

VITyarthi
Build Your Own Project

Expense Tracker

A modular command-line budget and spending analyzer built in Python

Student name	TANMAY CHACHAN
Registration number	26BCE10726
Course name and code	Introduction to Problem Solving and Programming [CSE1021]
Faculty	Dr. Rudrakalyan Nayak
Submission date	29/09/2026
GitHub repository	[https://github.com/tanmayaggarwal12/python-expense-tracker]

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Keeping track of money sounds easy until you try to do it. Most of us know roughly what we spend, but we rarely add it up, and it is only at the end of the month that we notice the budget is gone. This project is a small Python tool that makes that check quick. You type in a budget, list where your money went and how much, and it shows the total, the average, your biggest and smallest expense, and whether you are still inside your limit.

I started with two files. `expense_tracker.py` held three functions, `set_global_budget()`, `spent_analysis()` and `loop_spent_analysis()`, and `main.py` called them. Everything worked, but `spent_analysis()` was doing all the work in one long function: reading input, calculating, printing and checking the budget. For this project I kept the same functions, the same logic and the same messages, and split the code across files so that each file has one job.

The project applies the core ideas from the course: functions, modular design, lists, loops and conditionals, exception handling and string handling.

3. Problem Statement

Students and first-time earners usually do not know where their money goes. Full budgeting apps and spreadsheets do solve this, but they need setup, accounts or regular upkeep, so many people never start. There is a gap for something very light: a tool that answers *how much did I spend, where did most of it go, and am I within my budget?* in under a minute.

Objectives

- Let the user set a budget and check spending against it.
- Accept several expenses at once with minimal typing.
- Report total, average, highest and lowest expense.
- Handle bad input with a clear message instead of crashing.
- Organise the code into separate, focused modules.

Scope

Included: budget setting, bulk expense entry, analysis, budget comparison, input handling and repeated rounds, with all data kept in memory only. Not included: saving data between runs, decimal amounts, multi-word categories, dates or monthly reports, and any graphical interface. These are listed under Future Enhancements.

4. Functional Requirements

The system is organised into three major functional modules (Budget Management, Expense Input, and Analysis and Reporting), plus the workflow that connects them.

ID	Module	Requirement
FR1	Budget Management	The system shall ask the user for a budget and accept only a number. Anything else shows an error and ends the round.
FR2	Budget Management	The system shall compare the total spent with the budget and show either the remaining money or an exceeded warning.

ID	Module	Requirement
FR3	Expense Input	The system shall accept space-separated area names and space-separated amounts in two prompts.
FR4	Expense Input	The system shall show 'Invalid input' if the two lists are empty or have different lengths, or if an amount is not a whole number.
FR5	Analysis and Reporting	The system shall calculate the total, average, highest and lowest expense.
FR6	Analysis and Reporting	The system shall print the total, the spent list, the average, the highest and the lowest, using the rupee symbol.
FR7	Workflow	The system shall run one round straight away, then ask whether to run another (yes/y) or quit (anything else, such as no/n).
FR8	Data handling	The system shall keep all data in memory while it runs and shall not save anything to a file or database.

Input / output structure

Input	Format	Example
Budget	A number	5000
Areas	Words separated by spaces	food travel rent
Amounts	Whole numbers separated by spaces, same count as areas	800 250 3000
Menu choice	yes / y to continue; anything else quits	y

Output	Description
Total / average	Sum and mean of all amounts
Highest / lowest	Largest and smallest single expense
Spent list	The areas and amounts entered
Budget status	Remaining money, or a warning that the budget was exceeded
Error message	'Invalid Input' / 'Invalid input' when something cannot be used

5. Non-functional Requirements

Requirement	What it means here	How it is met
Usability	A first-time user should understand the prompts without instructions.	Prompts show the expected format, for example 'Write the area of spent with spaces'.
Reliability	Bad input must not crash the program.	Invalid budgets, mismatched lists and non-numeric amounts are all caught and answered with a message.
Maintainability	Each file should have one job and be easy to change.	Six Python files, short functions, comments and docstrings. The original function names are kept.

Requirement	What it means here	How it is met
Error handling strategy	Errors are caught close to where they happen.	try/except ValueError around the budget and around the analysis; a check on list lengths before any calculation.
Privacy / simplicity	No personal spending data should be left behind.	All data lives in variables and lists in memory and is discarded when the program ends. No files or database are used.
Performance / efficiency	Results should appear instantly and use almost no memory.	One pass over a small list, held in memory; no external libraries.
Portability	It should run anywhere Python runs.	Standard library only, Python 3.8 or newer.

6. System Architecture

The application is layered in a simple way. **main.py** is the entry point. It calls the workflow in **app/expense_tracker.py**, which in turn calls four worker modules, one for each step: **budget**, **expense_input**, **analysis** and **report**. All data is kept in memory (variables and lists) while the program runs. Keeping each step in its own file means a change to one step, for example the report layout, does not touch the others.

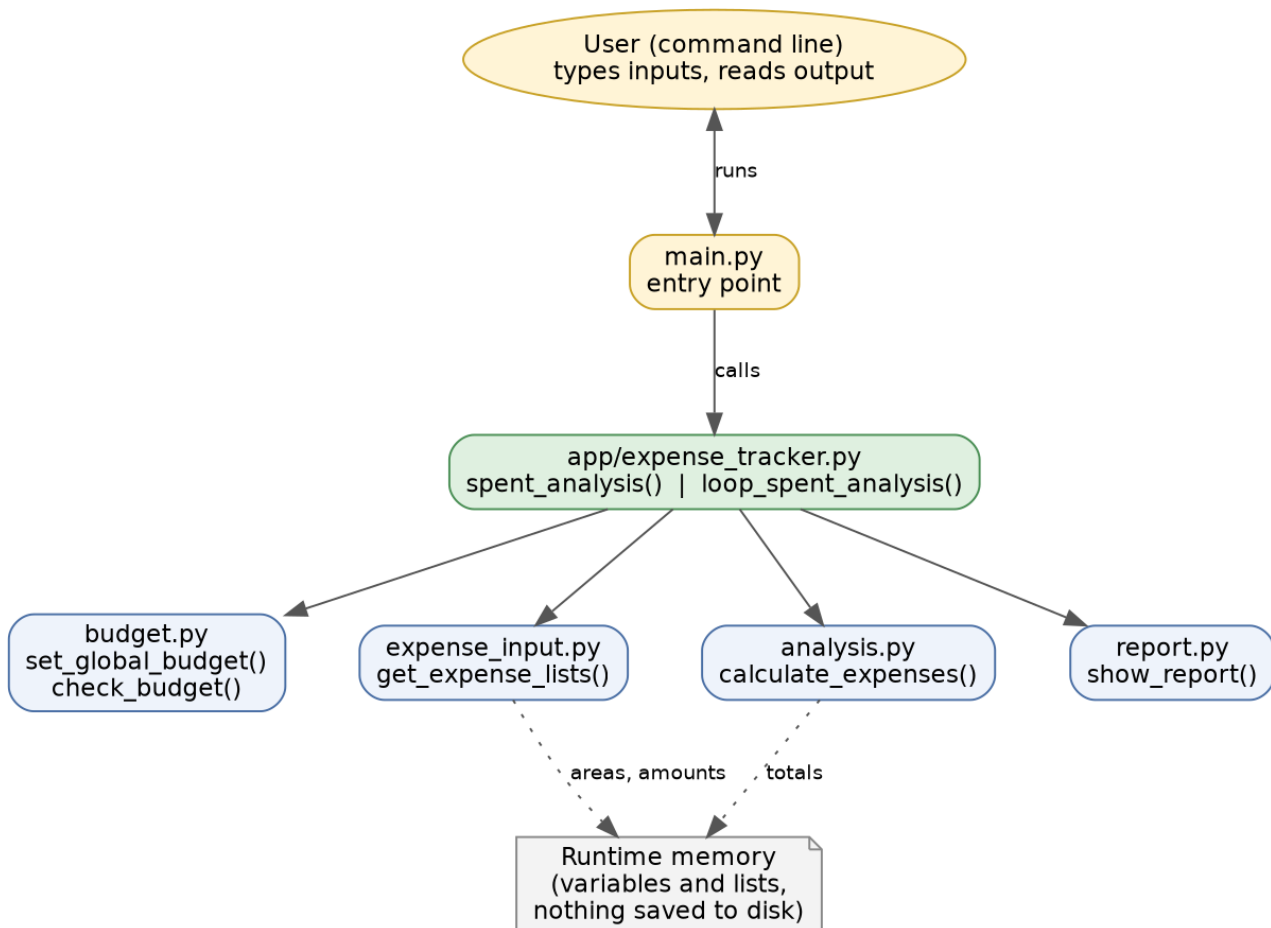


Figure 1: System architecture

Module overview

File	Responsibility
main.py	Entry point. Prints the banner, runs the first round, then starts the yes/no loop.
app/expense_tracker.py	Workflow: spent_analysis() runs one round, loop_spent_analysis() repeats rounds.
app/budget.py	set_global_budget() reads the budget; check_budget() compares it with the total.
app/expense_input.py	get_expense_lists() reads the areas and amounts and checks the lists match.
app/analysis.py	calculate_expenses() works out total, average, highest and lowest.
app/report.py	show_report() prints the results.

7. Design Diagrams

7.1 Use Case Diagram

There is one actor, the user. Input validation is shown as included behaviour because it runs automatically as part of other use cases.

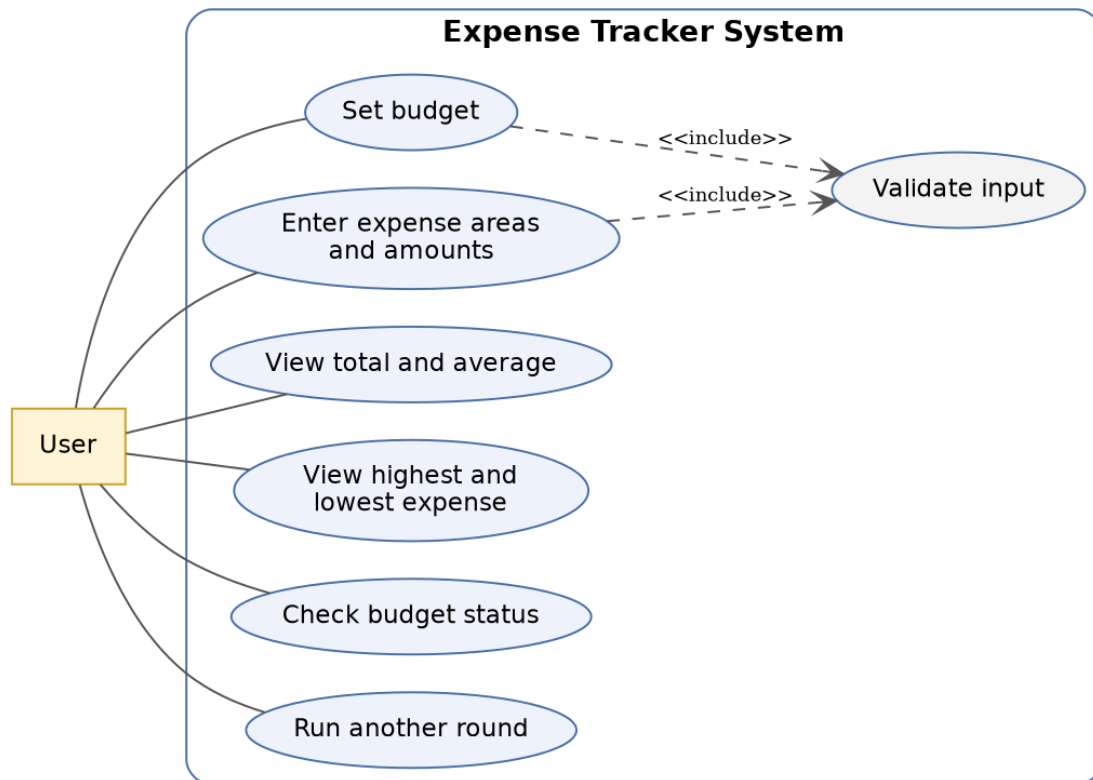


Figure 2: Use case diagram

7.2 Workflow Diagram

The program runs one round straight away and then keeps asking whether to run another. Any invalid input ends the current round with an error message and returns to that question.

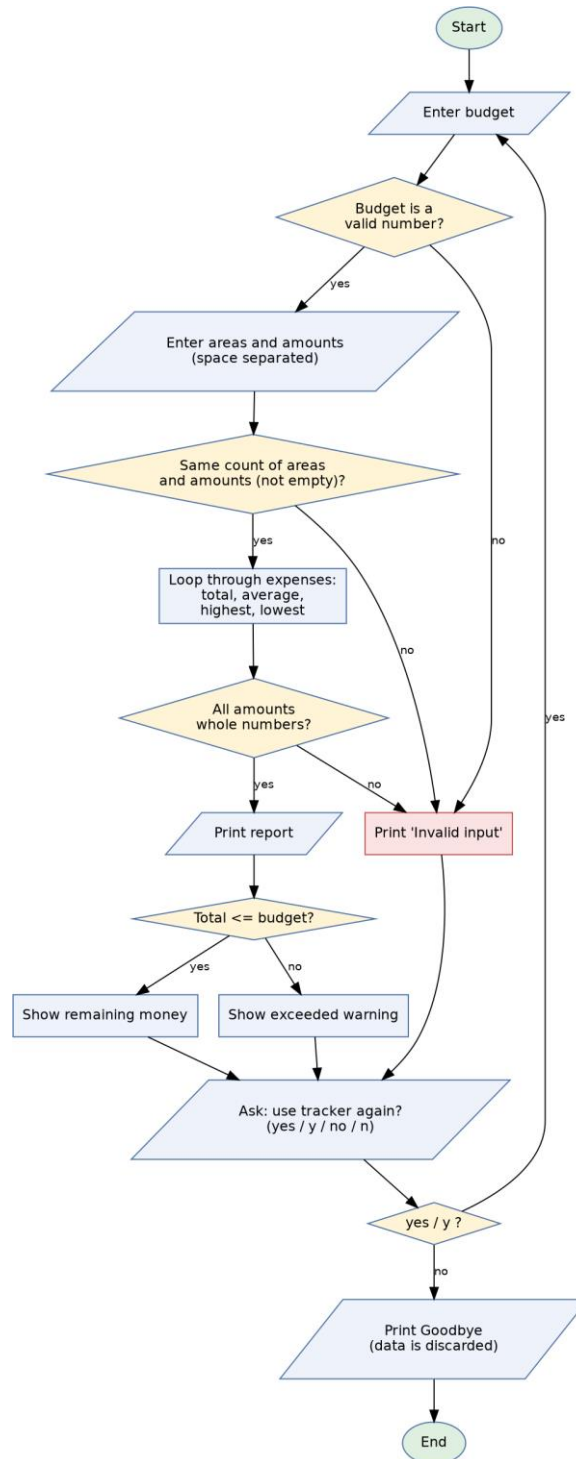


Figure 3: Workflow of the program

7.3 Sequence Diagram

This shows a successful round, followed by the question about running again.

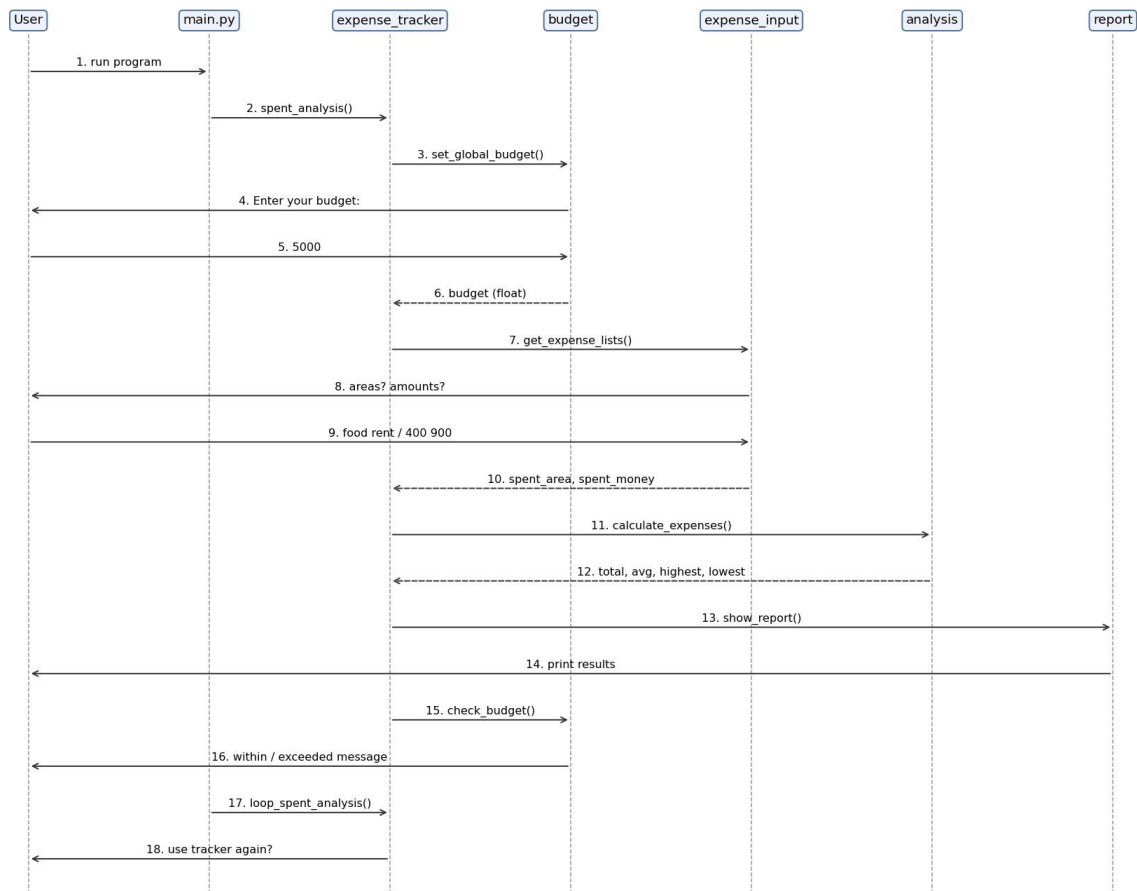


Figure 4: Sequence diagram for a successful round

7.4 Component Diagram

The code is written with functions rather than classes, so a component diagram fits better than a class diagram. Each box is a Python file, with the functions it provides and the value each one returns.

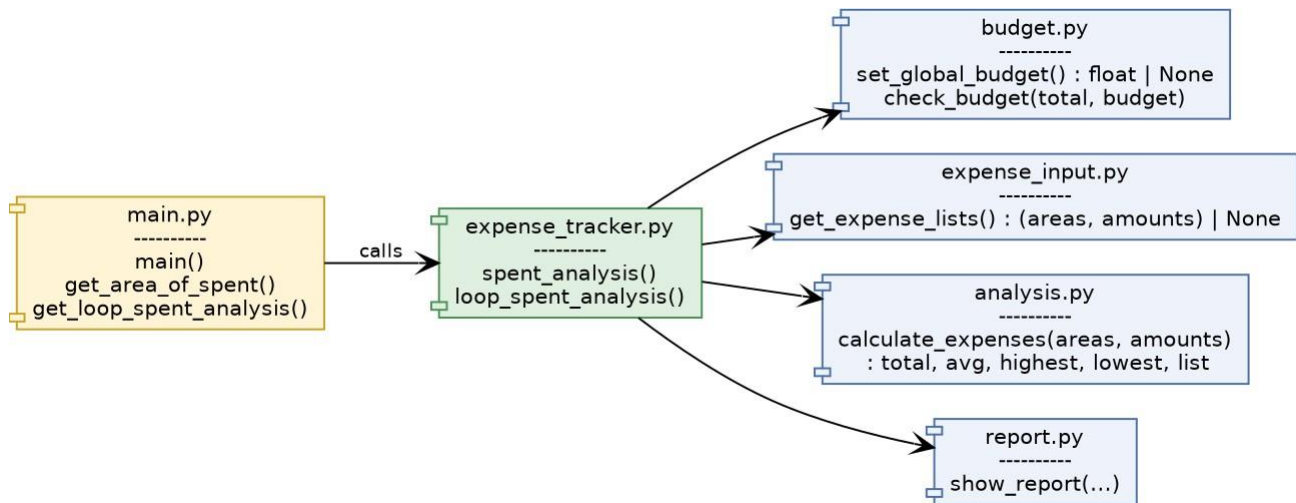


Figure 5: Component diagram

7.5 ER Diagram

Not applicable. The program does not store data between runs, so there is no database or schema. The data (the areas, amounts and totals) only lives in variables and lists while the program is running. Saving expenses is listed under Future Enhancements, and an ER diagram would be added at that point.

8. Design Decisions and Rationale

Decision	Why
Keep the original function names and logic	The project is built on my own first version. Keeping <code>set_global_budget()</code> , <code>spent_analysis()</code> and <code>loop_spent_analysis()</code> shows clearly which part of the old code went where.
Split by responsibility into six files	One long function was doing input, calculation, printing and the budget check. Separate files make each step easier to read and change, and meet the modular-implementation expectation.
Functions instead of classes	The program has no state to carry between steps, only values passed from one function to the next. Plain functions are simpler and match the course level.
Functions return values; the workflow decides what to do	<code>calculate_expenses()</code> returns its results instead of printing them, so the calculation is separate from the report. Functions that fail return <code>None</code> , and the workflow stops the round.
Guard against an invalid budget	In the original code, a bad budget made <code>set_global_budget()</code> return nothing and the later comparison crashed. The workflow now checks for <code>None</code> and ends the round with a message.
Check for empty input	Pressing Enter twice used to give a total of 0 and 'Lowest expense None'. An empty input is now treated as invalid.
Keep space-separated input	It is the original design and needs very little typing. The trade-off is that area names must be one word.
Keep whole-number amounts	The original converts amounts with <code>int()</code> . I kept it, so decimals are reported as invalid input. This is listed as a future improvement.
Keep print-based output	Matching the original messages keeps the behaviour familiar. The printing now lives in <code>report.py</code> and <code>budget.py</code> .
Runtime (in-memory) storage only	Data is kept in lists and variables while a round runs and is discarded when the program ends. This keeps the project simple and needs no database or files.
Standard library only	No install step for anyone who wants to run or mark the project.

9. Implementation Details

The full source is in the GitHub repository. This table shows where each part of the original code now lives.

Original code	Now in
<code>set_global_budget()</code> in <code>expense_tracker.py</code>	<code>app/budget.py</code> (same function, plus an explicit return <code>None</code>)
The two <code>input()</code> lines and <code>.split()</code> calls inside <code>spent_analysis()</code>	<code>app/expense_input.py</code> , <code>get_expense_lists()</code>
The for loop that adds up, finds highest/lowest and builds <code>expense_list</code>	<code>app/analysis.py</code> , <code>calculate_expenses()</code>
The block of <code>print()</code> lines showing total, list, average, highest, lowest	<code>app/report.py</code> , <code>show_report()</code>

Original code	Now in
The if total_sum <= global_bgt / else block	app/budget.py, check_budget()
spent_analysis() and loop_spent_analysis()	app/expense_tracker.py (same names; spent_analysis() now just calls the modules above)
main(), get_area_of_spent(), get_loop_spent_analysis()	main.py (same names)

Budget module (budget.py)

```

"""Budget module: asking for the budget and checking spending against it."""

def set_global_budget():
    """Ask the user for a budget. Returns the budget, or None if the input was invalid."""
    try:
        print("-" * 50)
        global_budget = float(input("Enter your budget: "))
        return global_budget
    except ValueError:
        print("Invalid Input")
        return None

def check_budget(total_sum, global_bgt):
    """Compare the total spent with the budget and tell the user the result."""
    if total_sum <= global_bgt:
        print("You are within your budget! Remaining money: ", "Rs.", global_bgt - total_sum)
    else:
        print("Warning: You have exceeded your budget!")

```

Expense input module (expense_input.py)

```

"""Expense input module: reads the areas and amounts typed by the user."""

def get_expense_lists():
    """Ask for the areas and the amounts (both separated by spaces).

    Returns (spent_area, spent_money) as two lists, or None if the input is invalid.
    The lists only exist in memory while the program is running.
    """
    area_of_spent = input("The area of spent [Write the area of spent with spaces]: ")
    spent_area = area_of_spent.split()

    money_spent = input("The amount of money spent [Write the expenditure with spaces]: ")
    spent_money = money_spent.split()

    # Both lists must have the same length and must not be empty
    if len(spent_area) == len(spent_money) and len(spent_area) > 0:
        return spent_area, spent_money

    print("Invalid input")
    return None

```

Analysis module (analysis.py)

This is the original loop, moved into its own function. It goes through the expenses once and keeps track of the total, highest and lowest as it goes.

```

"""Analysis module: total, average, highest and lowest expense."""

def calculate_expenses(spent_area, spent_money):

```

```

"""Go through the expenses once and work out the summary numbers.

Raises ValueError if any amount is not a whole number.
Returns total_sum, avg, highest, lowest, expense_list.
"""
expense_list = []
total_sum = 0
avg = 0
highest = 0
lowest = None

for i in range(len(spent_area)):
    current_expense = int(spent_money[i])
    total_sum = total_sum + current_expense

    if current_expense > highest:
        highest = current_expense
    if lowest is None or current_expense < lowest:
        lowest = current_expense

    expense_list.append(spent_area[i])
    expense_list.append(spent_money[i])

if len(spent_area) > 0:
    avg = total_sum / len(spent_area)

return total_sum, avg, highest, lowest, expense_list

```

Workflow (expense_tracker.py)

```

"""Main workflow: ties the budget, input, analysis and report modules together."""

from app import analysis, budget, expense_input, report

def spent_analysis():
    """One full round: budget -> expenses -> analysis -> report -> budget check."""
    try:
        global_bgt = budget.set_global_budget()
        if global_bgt is None:
            # invalid budget, stop this round
            return

        expense_lists = expense_input.get_expense_lists()
        if expense_lists is None:
            # invalid expenses, stop this round
            return
        spent_area, spent_money = expense_lists

        total_sum, avg, highest, lowest, expense_list = analysis.calculate_expenses(spent_area, spent_money)

        report.show_report(total_sum, expense_list, avg, highest, lowest)
        budget.check_budget(total_sum, global_bgt)
    except ValueError:
        print("Invalid input")

def loop_spent_analysis():
    """Keep offering another round until the user says no."""
    while True:
        choice = input("If you want to use expense tracker type yes or y or not want then type no or n ")

        if choice == "yes" or choice == "y":
            spent_analysis()
        else:
            print("Goodbye!")
            break

```

Concepts used

- **Functions and modules** for separation of concerns.
- **Lists** to hold the areas, the amounts and the expense list.
- **Loops and conditionals** for the analysis loop, the yes/no menu and the budget check.
- **Exception handling** (try/except ValueError) for invalid numbers.
- **String handling** with input() and split().

10. Screenshots / Results

These are real outputs from running `python main.py`.

Run 1: spending within budget

```
PS C:\Users\hp\Desktop\Vidarthi Project> & C:\Users\hp\AppData\Local\Programs\Python\Python315\python.exe "c:/Users/hp/Desktop/Vidarthi Project/expense-tracker/main.py"
----- Start Tracking Expenses -----
-----
Enter your budget: 5000
The area of spent [Write the area of spent with spaces]: food travel rent snacks
The amount of money spent [Write the expenditure with spaces]: 800 250 3000 150
*****
Total calculated: ₹ 4200
Spent_list: ['food', '800', 'travel', '250', 'rent', '3000', 'snacks', '150']
Average expense: ₹ 1050.0
Highest expense ₹ 3000
Lowest expense ₹ 150
You are within your budget! Remaining money: ₹ 800.0
If you want to use expense tracker type yes or y or not want then type no or n n
Goodbye!
PS C:\Users\hp\Desktop\Vidarthi Project> █
```

Total 4200 against a budget of 5000 leaves 800. The average is 1050, the highest expense is 3000 and the lowest is 150.

Run 2: budget exceeded

```
PS C:\Users\hp\Desktop\Vidarthi Project> & C:\Users\hp\AppData\Local\Programs\Python\Python315\python.exe "c:/Users/hp/Desktop/Vidarthi Project/expense-tracker/main.py"
----- Start Tracking Expenses -----
-----
Enter your budget: 1000
The area of spent [Write the area of spent with spaces]: food rent
The amount of money spent [Write the expenditure with spaces]: 400 900
*****
Total calculated: ₹ 1300
Spent_list: ['food', '400', 'rent', '900']
Average expense: ₹ 650.0
Highest expense ₹ 900
Lowest expense ₹ 400
Warning: You have exceeded your budget!
If you want to use expense tracker type yes or y or not want then type no or n n
Goodbye!
PS C:\Users\hp\Desktop\Vidarthi Project> █
```

Total 1300 against a budget of 1000, so the program warns that the budget was exceeded.

Run 3: invalid input

```
PS C:\Users\hp\Desktop\Vidyardhi Project> & C:\Users\hp\AppData\Local\Programs\Python\Python315\python.exe "c:/Users/hp/Desktop/Vidyardhi Project/expense-tracker/main.py"
----- Start Tracking Expenses -----
-----
Enter your budget: abc
Invalid Input
If you want to use expense tracker type yes or y or not want then type no or n y
-----
Enter your budget: 2000
The area of spent [Write the area of spent with spaces]: food travel
The amount of money spent [Write the expenditure with spaces]: 100
Invalid input
If you want to use expense tracker type yes or y or not want then type no or n y
-----
Enter your budget: 2000
The area of spent [Write the area of spent with spaces]: food travel
The amount of money spent [Write the expenditure with spaces]: 100 abc
Invalid input
If you want to use expense tracker type yes or y or not want then type no or n y
-----
Enter your budget: 2000
The area of spent [Write the area of spent with spaces]: food travel
The amount of money spent [Write the expenditure with spaces]: 100 250
*****
Total calculated: ₹ 350
Spent_list: ['food', '100', 'travel', '250']
Average expense: ₹ 175.0
Highest expense ₹ 250
Lowest expense ₹ 100
You are within your budget! Remaining money: ₹ 1650.0
If you want to use expense tracker type yes or y or not want then type no or n n
Goodbye!
```

The first round has a text budget, so the program prints 'Invalid Input' and goes to the yes/no question. The next rounds use mismatched counts and then a non-numeric amount, and each gets 'Invalid input'. The last round is valid and works normally.

11. Testing Approach

The project was tested manually by running the program with different kinds of input and comparing the output with results worked out by hand. The real outputs are shown in Section 10. Each function does one job, so it is easy to see which step is responsible when a result is wrong.

Test case	Input	Expected result	Result
Normal run, within budget	Budget 5000; food travel rent snacks; 800 250 3000 150	Total 4200, average 1050.0, highest 3000, lowest 150, remaining 800.0	Passed
Budget exceeded	Budget 1000; food rent; 400 900	Total 1300, exceeded warning	Passed
Exactly on budget	Budget 500; food; 500	Within budget, remaining 0.0	Passed
Budget is text	abc	'Invalid Input', then the yes/no question	Passed
Different counts	Areas: food travel; amounts: 100	'Invalid input', then the yes/no question	Passed

Test case	Input	Expected result	Result
Non-numeric amount	Amounts: 100 abc	'Invalid input', then the yes/no question	Passed
Decimal amount	Amounts: 100 250.5	'Invalid input' (whole numbers only, by design)	Passed
Empty input	Press Enter for areas and amounts	'Invalid input'	Passed
Repeat and quit	y, yes, then n	y and yes start a new round; n prints Goodbye!	Passed

12. Challenges Faced

- **Crash when the budget was invalid.** If the budget was not a number, `set_global_budget()` printed a message and returned nothing. The code kept going and then failed comparing the total with nothing. I fixed it by returning `None` on purpose and having the workflow check for it.
- **One function doing everything.** `spent_analysis()` read the input, calculated, printed and checked the budget. It was hard to see where a problem started. Splitting it up meant deciding what each new function should take in and hand back.
- **Printing while calculating.** Because the calculation and the print statements were mixed together, the numbers could not be reused. Now `calculate_expenses()` returns them and `show_report()` prints them.
- **Empty input.** Pressing Enter at both prompts gave a total of 0 and 'Lowest expense None'. I added a check so empty input counts as invalid.
- **Deciding how to split a small program.** I had to make the five or more modules meaningful and not artificial. I split by step (budget, input, analysis, report and workflow) so that every file has a real reason to exist.
- **Keeping the original behaviour.** I wanted the same prompts and messages as my first version, so I had to move code without changing what it prints.
- **Known limits.** Space-separated input means 'eating out' becomes two areas, and `int()` means decimals are rejected. I kept them and documented them rather than complicating the input.

13. Learnings and Key Takeaways

- Small programs benefit from structure too. Splitting the code by responsibility made it much easier to read.
- A function that returns a value is more flexible than one that prints it.
- When a function can fail, decide clearly what it returns (here, `None`) and check for it. Otherwise the error shows up later, somewhere confusing.
- Trying awkward input (empty, text, mismatched counts) found problems that normal input never showed.
- Writing the requirements and diagrams after the code made me notice inconsistencies, such as the missing empty-input check.

14. Future Enhancements

- Accept decimal amounts (use float instead of int) and reject negative numbers.
- Save expenses to a file or SQLite database so history survives between runs (this would also need an ER diagram).
- Add dates and monthly or weekly summaries.
- Group repeated areas and show each area's share of the total.
- Allow multi-word category names, for example by using commas as separators.
- Export reports to CSV.
- Add simple charts, such as a bar chart of spending by area.
- Build a Tkinter or web interface.
- Set budget limits per category, not only overall.

15. References

- Python Software Foundation. *The Python Standard Library*. <https://docs.python.org/3/library/>
- Python Software Foundation. *PEP 8 - Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- Graphviz. *Graphviz documentation* (used to draw the architecture, workflow, use case and component diagrams). <https://graphviz.org/documentation/>
- Matplotlib. *Matplotlib documentation* (used to draw the sequence diagram). <https://matplotlib.org/stable/>
- VITyarthi. *Build Your Own Project: General Project Instructions and Submission Guidelines*.
- Course lecture material and notes.